Practical - 13

PHP JSON API and AJAX/Fetch-Based CRUD without Page Reload

A . Theory

Create simple PHP JSON API endpoints for events or students and consume them using Fetch API. Perform list, add, update, and delete operations asynchronously without full page reload.

1. Do PHP endpoints return valid JSON responses? Yes, PHP endpoints return valid JSON responses containing the result of the requested operation, along with appropriate messages.

2. Are GET and POST requests handled properly? Yes, GET requests are used to retrieve data, while POST requests are used to submit data to the server.

3. Are loading, success, and error states shown on the frontend? Yes, loading messages are shown while data is being processed, and success or error messages are displayed after the operation is completed.

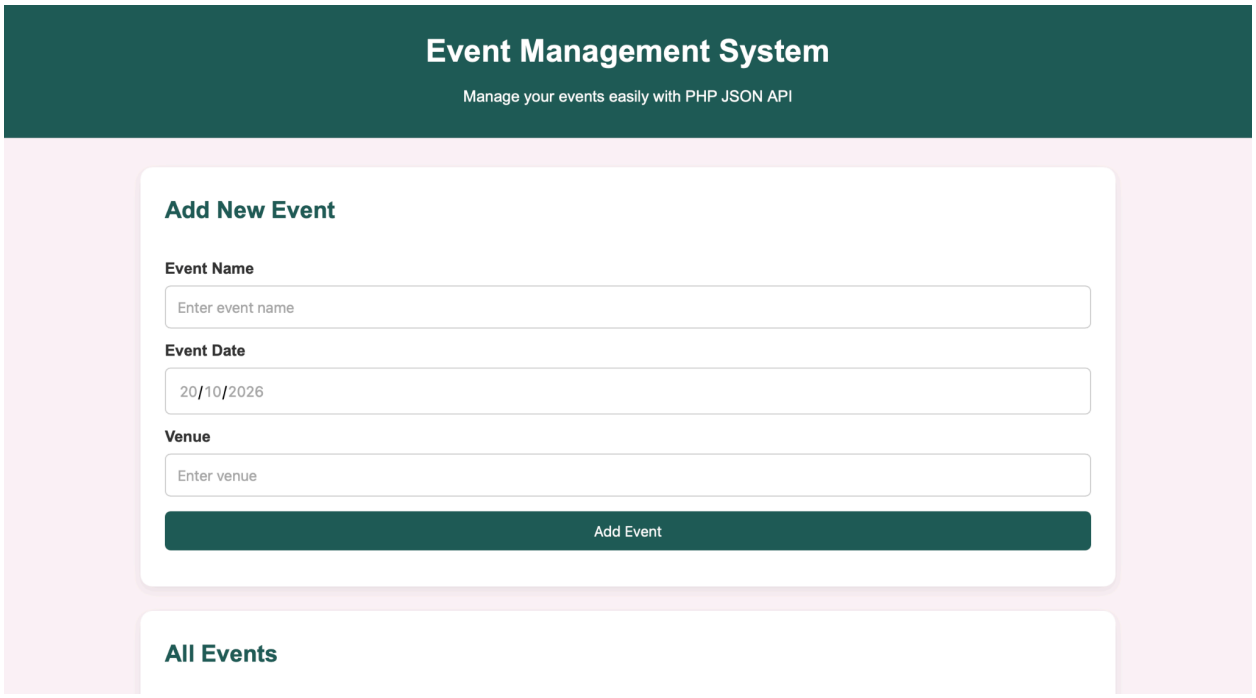
4. Is input validation performed before database operations? Yes, input validation is performed before processing data to prevent invalid or incomplete information from being stored in the database

B. Implementation/ Output/Result

Here is my GitHub repo link the codes are here in repo:
https://github.com/dhyaani19/StudentHub-Portal.git

C.. Evidence of Work (Screenshots)

Figure 13.1 : index page



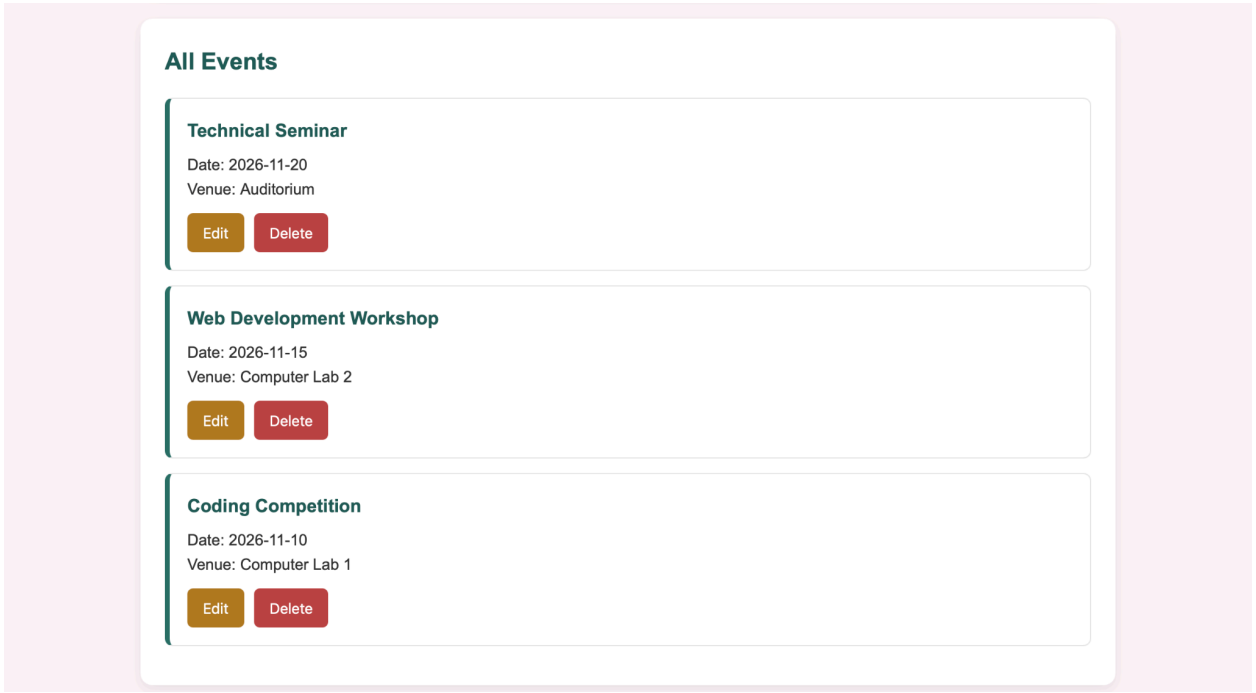


Figure 13.2 : Add Events

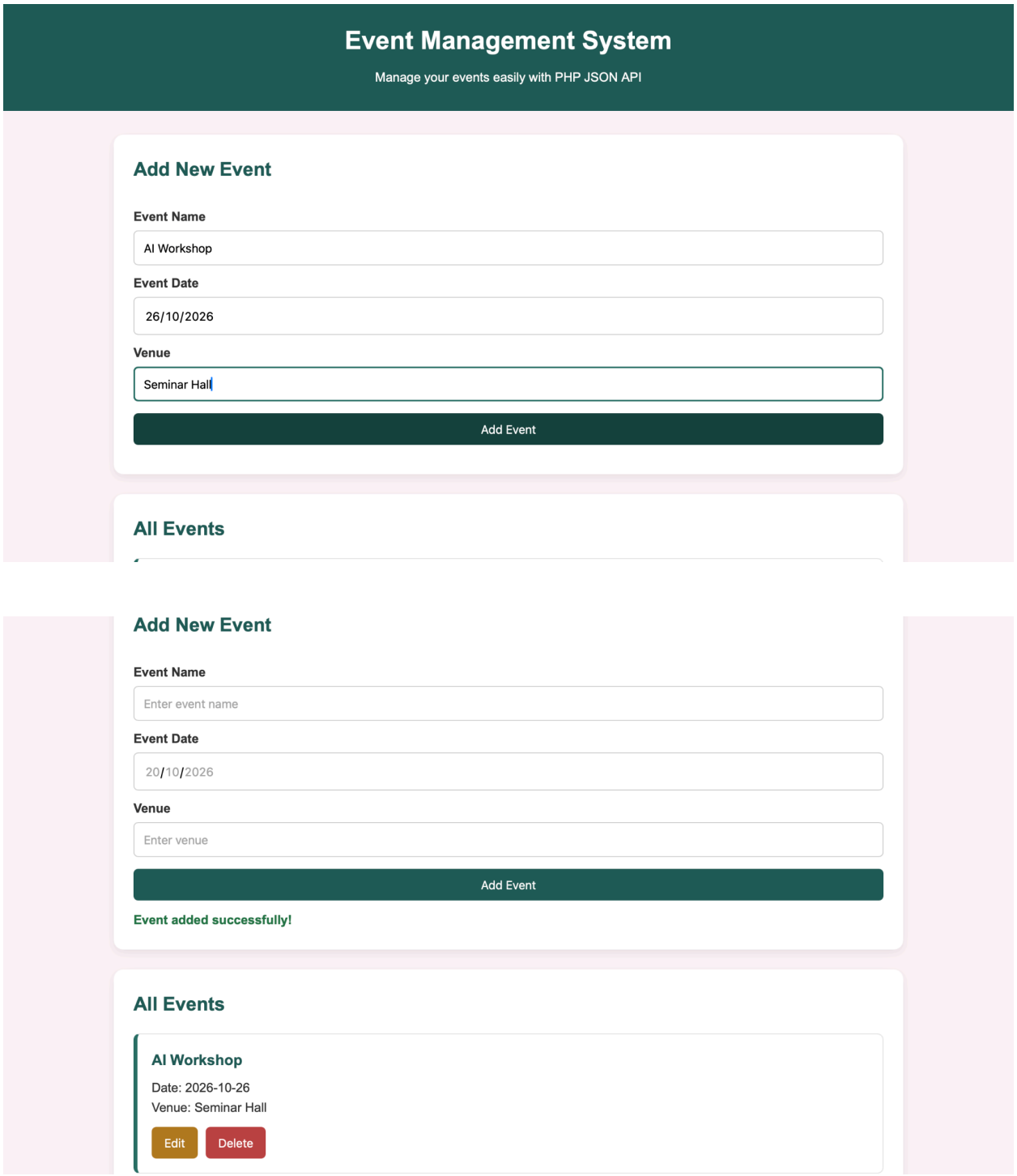


Figure 13.3 Update event

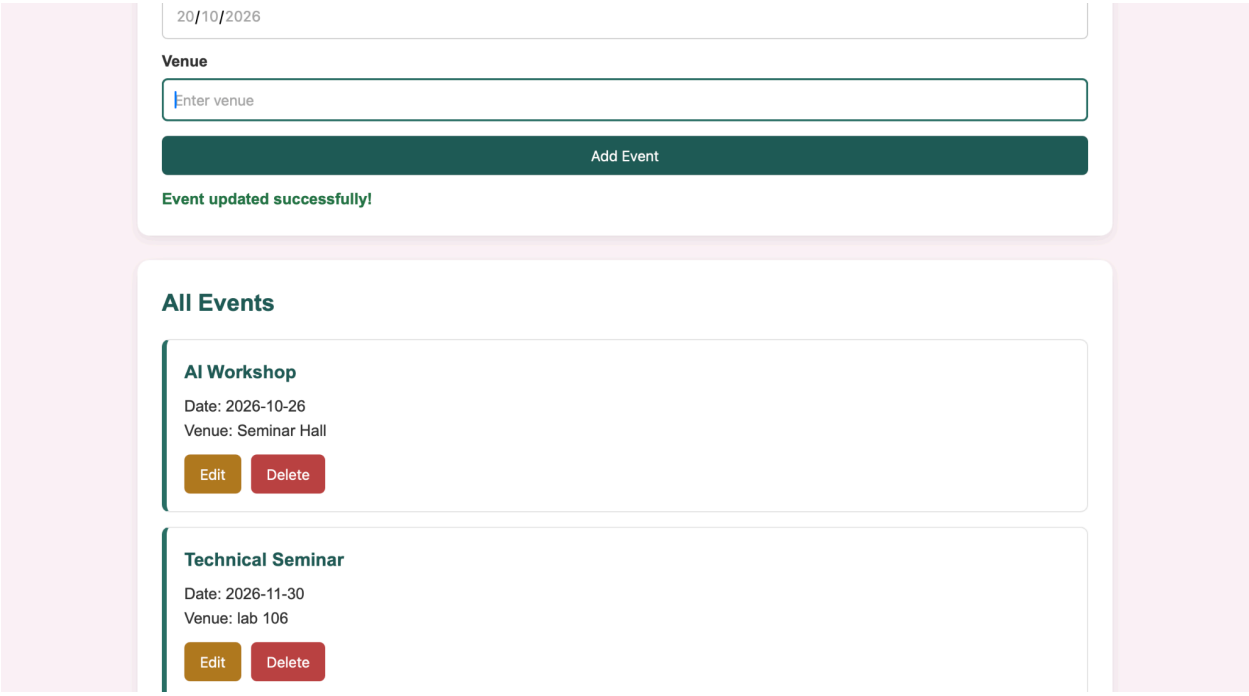
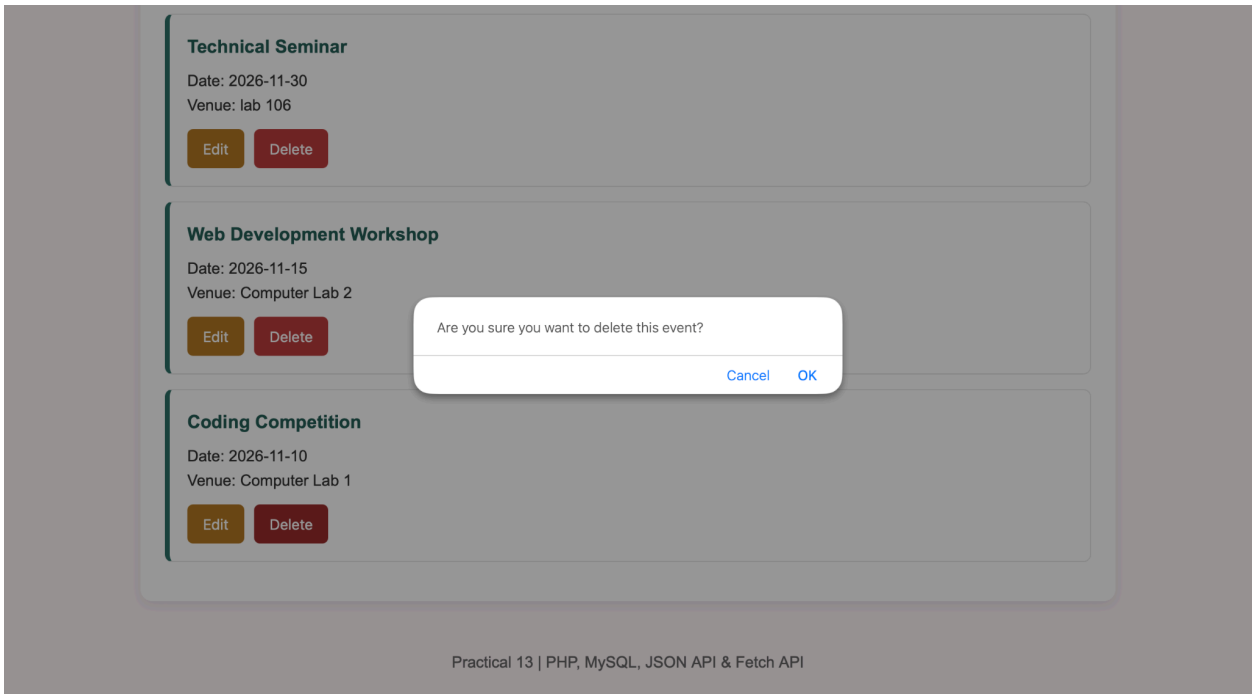
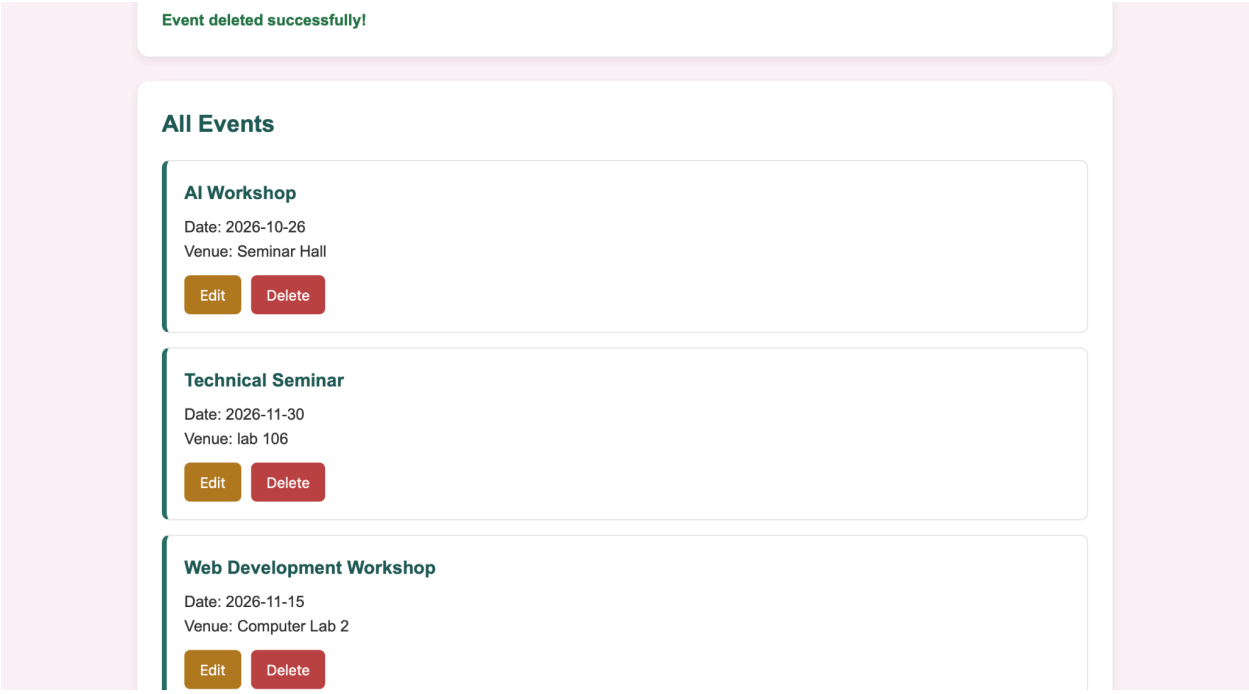


Figure 12.4 Delete event





D. GitHub Repository Link:

https://github.com/dhyaani19/StudentHub-Portal.git

E. Challenges Faced:

During this practical, I faced some difficulties in connecting PHP API endpoints with the frontend using the Fetch API. I had to understand how to send GET and POST requests and handle JSON data properly. Performing add, update, and delete operations without reloading the page was also a little challenging. I tested different operations and fixed errors to make sure the data was displayed and updated correctly.

F. Learning Outcomes:

Through this practical, I learned how to create PHP JSON API endpoints and connect them to a frontend using the Fetch API. I understood how to perform CRUD operations asynchronously without reloading the entire page. I also learned

how to handle GET and POST requests, convert responses into JSON, validate user input, and display appropriate success and error messages. This practical helped me understand how modern web applications communicate with the server.

Faculty Remarks: